Optimization Algorithms: Mini-Batch Gradient Descent

Motivation

- **Efficiency in Deep Learning:** Deep learning models perform best with very large datasets (Big Data).
- **The Problem with Batch Gradient Descent:**
  - Vectorization allows efficient processing of m examples, but when m is massive (e.g., 5 million), processing the entire training set for a single gradient descent step is too slow.
  - You must wait for the entire dataset to be processed before the parameters (W, b) are updated even once.
- **The Solution: Mini-Batch Gradient Descent** splits the training set into smaller "baby" training sets (mini-batches) to allow the model to start learning (updating parameters) before finishing the entire dataset.

Notation and Definitions

To manage the different subsets of data, specific notation is introduced:

- $X^{\{t\}}, Y^{\{t\}}$: Denotes the t-th mini-batch of input and output data.
- **Dimensions:**
  - If the total training set X is (n_x, m) and the mini-batch size is 1,000:
  - $X^{\{t\}}$ dimension: $(n_x, 1000)$.
  - $Y^{\{t\}}$ dimension: $(1, 1000)$.
- **Notation Summary:**
  - $x^{(i)}$: The i-th individual training example.
  - $z^{[l]}$: The value for the l-th layer of the neural network.
  - $X^{\{t\}}$: The t-th mini-batch of data.

Batch vs. Mini-Batch Gradient Descent

- **Batch Gradient Descent:**
  - Processes the **entire** training set at once.
  - Result: 1 gradient descent step per pass through the data.
- **Mini-Batch Gradient Descent:**
  - Processes a **single mini-batch** ($X^{\{t\}}, Y^{\{t\}}$) at a time.
  - Result: Multiple gradient descent steps per pass through the data.
  - *Example:* With 5,000,000 examples and a batch size of 1,000, there are 5,000 mini-batches. This allows 5,000 parameter updates per pass.

Definition: Epoch

- **Epoch**: A single pass through the **entire** training set.
  - In Batch Gradient Descent: 1 Epoch = 1 Gradient Descent Step.
  - In Mini-Batch Gradient Descent: 1 Epoch = $\frac{m}{\text{mini-batch size}}$ Gradient Descent Steps (e.g., 5,000 steps).

Algorithm: Mini-Batch Gradient Descent

The algorithm iterates through the mini-batches to perform updates.

Parameters:

- Total mini-batches: 5,000 (derived from $m = 5,000,000$ and batch size $= 1,000$).

Pseudocode:
For $t = 1$ to 5,000:

1. **Forward Propagation:**
   - Perform forward prop on $X^{\{t\}}$.
   - Compute activations: $Z^{[1]} = W^{[1]}X^{\{t\}} + b^{[1]}$, $A^{[1]} = g^{[1]}(Z^{[1]})$, ..., $A^{[L]}$.
   - *Note:* Use vectorization (processing 1,000 examples simultaneously).
2. **Compute Cost ($J^{\{t\}}$):**
   - Calculate cost strictly for the current mini-batch:
     $J^{\{t\}} = \frac{1}{1000} \sum_{i=1}^{l} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum ||W^{[l]}||_F^2$
3. **Backpropagation:**
   - Compute gradients with respect to $J^{\{t\}}$ using only ($X^{\{t\}}, Y^{\{t\}}$).
4. **Parameter Update:**

- Update weights and biases:
  $W^{[l]} := W^{[l]} - \alpha dW^{[l]}$
  $b^{[l]} := b^{[l]} - \alpha db^{[l]}$

Repeated passes (Epochs) are usually required until convergence.

Understanding Mini-Batch Gradient Descent

Cost Function Behavior

When plotting the cost function to monitor optimization progress, the behavior differs significantly depending on the gradient descent method used.

- **Batch Gradient Descent:**
  - Computes cost over the entire training set.
  - **Expectation:** Cost should decrease on **every** single iteration.
  - *Diagnostic:* If the cost goes up, something is likely wrong (e.g., learning rate is too large).
- **Mini-Batch Gradient Descent:**
  - Computes cost $J^{\{t\}}$ using only the current mini-batch $(X^{\{t\}}, Y^{\{t\}})$.
  - **Expectation:** The plot will be **noisy**. It should trend downwards over time, but will oscillate.
  - *Reason for Noise:* Not all mini-batches are equally difficult. Some may contain "easy" examples (cost goes down), while others may contain "hard" or mislabeled examples (cost goes up temporarily).

The Spectrum of Batch Sizes

The size of the mini-batch is a hyperparameter that defines the algorithm's behavior. Let m be the training set size.

1. Batch Gradient Descent (Size = m)

- **Definition:** The mini-batch size equals the entire training set size.
- **Behavior:**
  - Low noise, takes relatively large steps directly toward the minimum.

- **Disadvantage:** Too slow per iteration if the dataset is large (e.g., millions of examples). You must process all data before taking a single step.

2. Stochastic Gradient Descent (SGD) (Size = 1)

- **Definition:** Every single example is its own mini-batch.
- **Behavior:**
  - Extremely noisy; the gradient approximation relies on a single example.
  - **Oscillation:** It will wander around the region of the minimum but will never strictly converge.
- **Disadvantage:** Loses the speedup benefits of **vectorization**. Processing examples one by one is computationally inefficient.

3. Mini-Batch Gradient Descent (Size between 1 and m)

- **Definition:** A compromise between the two extremes (e.g., size = 64, 128, etc.).
- **Benefits:**
  - **Vectorization:** Unlike SGD, you can process a chunk of data (e.g., 1,000 examples) efficiently in parallel.
  - **Speed:** Unlike Batch, you make progress (update parameters) without waiting to process the entire dataset.
- **Convergence:** It tends to head toward the minimum more consistently than SGD but will still oscillate in a small region around the minimum. (This can be resolved by decaying the learning rate).

Guidelines for Choosing Mini-Batch Size

The mini-batch size is a hyperparameter that should be tuned.

Decision Rules

1. **Small Training Set ($m \leq 2,000$):**
   - Use **Batch Gradient Descent**.
   - With small data, the overhead of mini-batches isn't worth it; processing the whole batch is fast enough.
2. **Large Training Set:**
   - Use **Mini-Batch Gradient Descent**.
   - **Typical Sizes:** 64, 128, 256, 512.
   - **Power of 2 Rule:** It is highly recommended to use powers of 2 (e.g.,

$2^6, 2^7, 2^8$) because computer memory is laid out in a way that often makes these specific sizes run faster.

- *Note:* Sizes like 1,024 are possible but less common.

Hardware Constraints

- **CPU/GPU Memory:** Ensure the chosen mini-batch size fits within your processor's memory.
- If the batch is too large for memory, performance "falls off a cliff" due to swapping or thrashing.

Search Strategy

- Treat mini-batch size as a hyperparameter to optimize.
- Try different powers of 2 (64, 128, 256, etc.) and observe which setting reduces the cost function J most efficiently.

Optimization Algorithms: Exponentially Weighted Averages

Introduction

- **Purpose:** Exponentially Weighted Averages (also known as Exponentially Weighted Moving Averages in statistics) are a foundational concept required to understand sophisticated optimization algorithms that are faster than standard Gradient Descent.
- **Goal:** To compute a local average or "trend" within a noisy dataset.

The Algorithm

Formula

The weighted average v_t at time step t is calculated recursively:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

Where:

- v_t: The exponentially weighted average at time t.
- v_{t-1}: The average from the previous time step.

- θ_t: The actual observed value at time t (e.g., current temperature).
- β: A hyperparameter ($0 < \beta < 1$) that controls the "window" of the average.

Interpretation of β

- v_t can be interpreted as approximately averaging over $\frac{1}{1-\beta}$ days.
- **The trade-off:** β controls the balance between smoothness (noise reduction) and latency (responsiveness to recent changes).

Example: Daily Temperatures

Consider a dataset of daily temperatures in London over a year ($\theta_1, \theta_2, \ldots, \theta_{365}$).

Case 1: $\beta = 0.9$ (Red Line)

- **Window Size:** $\frac{1}{1-0.9} = 10$ days.
- **Behavior:** This generates a smooth curve that represents the average temperature over roughly the last 10 days. It effectively captures the trend without excessive noise or lag.

Case 2: $\beta = 0.98$ (Green Line)

- **Window Size:** $\frac{1}{1-0.98} = 50$ days.
- **Behavior:**
  - **Pros:** Much smoother curve; filters out almost all noise.
  - **Cons:** Higher **latency**. The curve shifts to the right because it relies heavily on data from the distant past (0.98 weight on previous state). It adapts very slowly to temperature changes.

Case 3: $\beta = 0.5$ (Yellow Line)

- **Window Size:** $\frac{1}{1-0.5} = 2$ days.
- **Behavior:**
  - **Pros:** Adapts extremely quickly to changes.
  - **Cons:** Very **noisy** and susceptible to outliers. It closely follows the raw data rather than showing a stable trend.

Summary of Effects

Beta (β)	Approx. Window ($\frac{1}{1-\beta}$)	Effect on Curve
0.98 (High)	~50 days	Very smooth, high latency (slow to adapt).
0.90 (Medium)	~10 days	Balanced smoothness and responsiveness.
0.50 (Low)	~2 days	Noisy, low latency (fast to adapt).

Understanding Exponentially Weighted Averages

Mathematical Intuition

To understand how the formula $v_t = \beta v_{t-1} + (1-\beta)\theta_t$ computes an average, we can expand the recursive equation.

Example Expansion ($\beta = 0.9$):
Let's analyze the value at $t = 100$:

1. $v_{100} = 0.1\theta_{100} + 0.9v_{99}$
2. Substitute v_{99}: $v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98})$
3. Substitute v_{98}: $v_{100} = 0.1\theta_{100} + 0.1 \cdot 0.9\theta_{99} + 0.1 \cdot (0.9)^2\theta_{98} + \dots$

General Summation Form:
$v_t = \sum_{k=0}^{t}(1-\beta)\beta^k\theta_{t-k}$

Interpretation:

- v_t is a sum of current and past observations θ.
- **Element-wise Product:** You are taking the list of daily temperatures and performing an element-wise multiplication with an **exponentially decaying function** (the weights).
- **Weights:**
    - Current day (t): weight is $(1-\beta)$.

    - Yesterday ($t-1$): weight is $(1-\beta)\beta$.
    - k days ago: weight is $(1-\beta)\beta^k$.
- Since $\beta < 1$, the terms β^k get smaller as k increases, giving less weight to older data.

The "Window Size" Rule of Thumb

Why do we say v_t averages over approximately $\frac{1}{1-\beta}$ days?

Mathematical Derivation:

- We want to know how far back we go before the weight becomes negligible.
- A common standard for "negligible" is when the weight decays to $\frac{1}{e}$ (approx. 0.35 or ~1/3) of the peak.
- Using the limit property $(1-\epsilon)^{1/\epsilon} \approx \frac{1}{e}$:
    - Let $\epsilon = 1-\beta$.
    - Then $\beta^{1/(1-\beta)} \approx \frac{1}{e}$.

Examples:

- **If $\beta = 0.9$:** $(0.9)^{10} \approx 0.35$.
    - It takes **10 days** for the weight to decay to about 1/3.
    - Window size $\approx \frac{1}{1-0.9} = 10$.
- **If $\beta = 0.98$:** $(0.98)^{50} \approx 0.36$.
    - It takes **50 days** for the weight to decay to about 1/3.
    - Window size $\approx \frac{1}{1-0.98} = 50$.

Implementation and Efficiency

While standard moving averages (summing the last N days and dividing by N) are mathematically more precise for a fixed window, Exponentially Weighted Averages are preferred in Deep Learning for their efficiency.

Comparison

Feature	Standard Moving Average	Exponentially Weighted Average
Memory	High (must store last N values).	**Extremely Low** (stores only 1 number).

Feature	Standard Moving Average	Exponentially Weighted Average
Compute	Higher (summing list).	**Very Low** (1 multiply, 1 add).
Implementation	Complex buffer management.	**1 Line of Code.**

Algorithm Code

1. **Initialize:** `v = 0`
2. **Iterate:** For each new data point `theta`:

```
v = beta * v + (1 - beta) * theta
```

Optimization Algorithms: Bias Correction in Exponentially Weighted Averages

The Problem: Initialization Bias

When implementing exponentially weighted averages, we initialize $v_0 = 0$. This initialization causes a significant error during the initial phase of learning (the "warm-up" period), resulting in estimates that are much lower than the actual data.

- **Visual Representation:**
  - **Green Curve:** The ideal moving average.
  - **Purple Curve:** The actual output without bias correction (starts extremely low and slowly climbs to meet the green curve).

Mathematical Example

Assume $\beta = 0.98$ and the first observation $\theta_1 = 40$.

1. **Step 1:**
   $v_1 = 0.98v_0 + 0.02\theta_1$
   Since $v_0 = 0$:
   $v_1 = 0.98(0) + 0.02(40) = 0.8$

Result: The estimate is 0.8, which is a very poor representation of the actual temperature 40.

2. **Step 2:**
   $v_2 = 0.98v_1 + 0.02\theta_2$
   $v_2 = 0.98(0.02\theta_1) + 0.02\theta_2 = 0.0196\theta_1 + 0.02\theta_2$
   Result: The weights ($0.0196 + 0.02 = 0.0396$) sum to far less than 1, causing the weighted average to be drastically smaller than θ_1 and θ_2.

The Solution: Bias Correction

To fix this, we introduce a scaling factor that compensates for the initial zero values.

Corrected Formula:
$\text{Corrected } v_t = \frac{v_t}{1-\beta^t}$

Mechanism

- **During Initial Phase (Small t):**
  - When $t = 2$ and $\beta = 0.98$, the denominator is $1 - 0.98^2 = 0.0396$.
  - Scaling v_2 by $\frac{1}{0.0396}$ normalizes the weights so they sum to 1.
  - Result: The estimate becomes an accurate weighted average of the early data points.
- **During Later Phase (Large t):**
  - As t increases, β^t approaches 0.
  - The denominator $1 - \beta^t$ approaches 1.
  - Result: The bias correction naturally fades away, and the curve converges to the standard exponentially weighted average (Purple line overlaps with Green line).

Practical Application in Machine Learning

- **Standard Practice:** Many ML implementations omit bias correction. Users often accept that the first few iterations (epochs) will be skewed while the moving average "warms up."
- **Recommendation:** If accurate estimates during the very early stages of training are critical for your application, you should implement bias correction.

Optimization Algorithms: Gradient Descent with Momentum

Overview

- **Concept:** Momentum is an optimization algorithm that almost always converges faster than standard Gradient Descent.
- **Mechanism:** It computes an **exponentially weighted average** of the gradients and uses this average to update the weights, rather than the raw gradients from the current step.
- **Intuition:**
  - **Standard Gradient Descent:** Can oscillate heavily (like a zigzag path) towards the minimum. These oscillations slow down learning and prevent the use of a large learning rate (which would cause divergence).
  - **Momentum:**
    - **Vertical Axis (Oscillations):** Positive and negative gradients from successive steps cancel each other out, damping the oscillations (average ≈ 0).
    - **Horizontal Axis (Towards Minimum):** Gradients consistently point in the same direction, so the average accumulates, leading to faster convergence.

Algorithm Implementation

Initialization

Initialize velocity terms v_{dW} and v_{db} to zero. These have the same dimensions as dW and db (and consequently W and b).

Update Steps (per iteration t)

1. **Compute Gradients:** Calculate dW, db on the current mini-batch.
2. **Compute Velocity (Moving Average):**
   $v_{dW} = \beta v_{dW} + (1 - \beta) dW$
   $v_{db} = \beta v_{db} + (1 - \beta) db$
   - *Note:* This applies the exponentially weighted average formula to the gradients.
3. **Update Parameters:**

$$W = W - \alpha v_{dW}$$
$$b = b - \alpha v_{db}$$

Key Hyperparameters:

- α **(Learning Rate):** Controls the step size.
- β **(Momentum):** Controls the exponentially weighted average window.
  - **Common Choice:** $\beta = 0.9$ (averages over roughly the last 10 gradients). This value is robust and works well in most cases.

Physics Analogy

- Imagine minimizing a cost function as a ball rolling down a bowl.
- **Derivatives (dW, db):** Act as **acceleration** (gravity pushing the ball down).
- **Velocity (v_{dW}, v_{db}):** Acts as the **velocity** of the ball.
- β **(Friction):** Acts as friction (since $\beta < 1$), preventing the ball from accelerating infinitely.
- Instead of taking independent steps, the ball gains "momentum" rolling downhill, smoothing out its path and speeding up.

Implementation Details & Variants

- **Bias Correction:** In practice, bias correction (dividing by $1 - \beta^t$) is **rarely used** for Momentum because the algorithm warms up quickly (after ~10 iterations).
- **Alternative Formula (Omitted $1 - \beta$):**
  - Some literature uses: $v_{dW} = \beta v_{dW} + dW$
  - *Effect:* This scales v_{dW} by a factor of $\frac{1}{1-\beta}$.
  - *Consequence:* The learning rate α must be retuned if β changes, making it less intuitive.
  - *Recommendation:* Use the standard formula (with $1 - \beta$) as it separates the tuning of β and α.

Optimization Algorithms: RMSprop

Overview

RMSprop (Root Mean Square Propagation) is an optimization algorithm designed to speed up gradient descent. similar to Momentum, it addresses the issue of oscillations, but it achieves this by adapting the learning rate for each parameter individually.

- **Goal:**
  - **Dampen oscillations** in directions with high variance (e.g., the vertical direction "b").
  - **Accelerate learning** in directions with low variance (e.g., the horizontal direction "w").
  - Allow the use of a **larger learning rate** (α) without divergence.

Intuition: Vertical vs. Horizontal

Consider a cost function with a steep slope in the vertical direction (b) and a gentle slope in the horizontal direction (w).

- **The Problem:** Standard gradient descent takes large steps in the steep direction (causing oscillations) and small steps in the gentle direction (causing slow convergence).
- **The RMSprop Solution:**
  - Calculate the square of the derivatives.
  - In the vertical direction (b), the derivatives (db) are large $\rightarrow db^2$ is very large.
  - In the horizontal direction (w), the derivatives (dW) are small $\rightarrow dW^2$ is small.
  - **Update Rule:** Divide the update step by the square root of this accumulated squared gradient.
    - **Vertical (b):** Divide by a large number $\rightarrow$ Small update step (damps oscillation).
    - **Horizontal (w):** Divide by a small number $\rightarrow$ Larger update step (speeds up learning).

Algorithm Implementation

1. Compute Gradients

Calculate dW and db on the current mini-batch.

2. Compute Exponentially Weighted Average of Squares

Compute a moving average of the **squared** gradients. Note that the squaring operation is **element-wise**. We use the notation S to denote this average.

$$S_{dW} = \beta_2 S_{dW} + (1 - \beta_2) dW^2$$
$$S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2$$

- β_2: Hyperparameter controlling the moving average window (distinct from the β used in Momentum).

3. Update Parameters

Update the weights and biases by dividing the gradient by the root mean square of the past gradients.

$$W := W - \alpha \frac{dW}{\sqrt{S_{dW}} + \epsilon}$$
$$b := b - \alpha \frac{db}{\sqrt{S_{db}} + \epsilon}$$

- ϵ **(Epsilon):** A very small number (e.g., 10^{-8}) added to the denominator to ensure **numerical stability** and prevent division by zero if S is close to 0.

Origin

Unlike most algorithms, RMSprop was not initially published in an academic paper. It was proposed by **Geoffrey Hinton** in a Coursera course similar to this one. It has since become a standard optimization method in Deep Learning.

Optimization Algorithms: Adam

Overview

- **Context:** While many optimization algorithms have been proposed in deep learning history, few generalize well to a wide range of neural networks.
- **Adam (Adaptive Moment Estimation):**
    - One of the rare algorithms that has proven effective across a vast array of

deep learning architectures.

- **Concept:** It combines the benefits of **Momentum** and **RMSprop** into a single algorithm.

Algorithm Implementation

Adam maintains two separate moving averages for each parameter:

1. V_{dw}: Exponentially weighted average of the **gradients** (derived from Momentum).
2. S_{dw}: Exponentially weighted average of the **squared gradients** (derived from RMSprop).

Initialization

Initialize $V_{dw} = 0, S_{dw} = 0, V_{db} = 0, S_{db} = 0$.

Iteration Steps (at time t)

1. **Compute Gradients:**
   Calculate dW, db on the current mini-batch.
2. **Update "Momentum" Terms (V):**
   - Compute the moving average of the gradients using hyperparameter β_1.
     $V_{dW} = \beta_1 V_{dW} + (1 - \beta_1) dW$
     $V_{db} = \beta_1 V_{db} + (1 - \beta_1) db$
3. **Update "RMSprop" Terms (S):**
   - Compute the moving average of the squared gradients using hyperparameter β_2.
     $S_{dW} = \beta_2 S_{dW} + (1 - \beta_2) dW^2$
     $S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2$
4. **Bias Correction:**
   - Unlike basic Momentum implementations, Adam typically implements bias correction to account for initialization at zero.
     $V_{dW}^{\text{corrected}} = \frac{V_{dW}}{1-\beta_1^t}, \quad V_{db}^{\text{corrected}} = \frac{V_{db}}{1-\beta_1^t}$
     $S_{dW}^{\text{corrected}} = \frac{S_{dW}}{1-\beta_2^t}, \quad S_{db}^{\text{corrected}} = \frac{S_{db}}{1-\beta_2^t}$
5. **Parameter Update:**
   - Update parameters using the corrected averages.
     $W = W - \alpha \frac{V_{dW}^{\text{corrected}}}{\sqrt{S_{dW}^{\text{corrected}}} + \epsilon}$
     $b = b - \alpha \frac{V_{db}^{\text{corrected}}}{\sqrt{S_{db}^{\text{corrected}}} + \epsilon}$

Hyperparameters

Adam requires tuning of four distinct hyperparameters.

Hyperparameter	Description	Typical/Recommended Value
α **(Alpha)**	Learning Rate.	**Must be tuned.** Try a range of values.
β_1 **(Beta 1)**	First Moment (Momentum) decay rate.	**0.9** (Default)
β_2 **(Beta 2)**	Second Moment (RMSprop) decay rate.	**0.999** (Default)
ϵ **(Epsilon)**	Numerical stability term (prevents division by zero).	10^{-8} (Rarely tuned).

Etymology

- **Name Origin: Ada**ptive **M**oment Estimation.
- **Mathematical Meaning:**
  - β_1 computes the mean of the derivatives (the **first moment**).
  - β_2 computes the variance (related to the **second moment**) of the derivatives.

Optimization Algorithms: Learning Rate Decay

Motivation

- **The Problem with Fixed Learning Rate:**
  - When using **Mini-Batch Gradient Descent**, the path to the minimum is noisy.
  - With a fixed learning rate α, the algorithm will wander or oscillate around the minimum but never truly converge.
- **The Solution (Decay):**

- **Initial Phase:** Start with a large α to take big steps and approach the minimum quickly.
- **Later Phase:** Slowly reduce (decay) α over time. This forces the algorithm to take smaller steps, allowing it to oscillate in a much tighter region around the minimum, effectively converging closer to the optimal solution.

Implementation Methods

Learning rate decay can be implemented using various formulas dependent on the **epoch number** (one full pass through the training data).

1. Inverse Time Decay (Standard)

This is a common formula used to gradually lower the rate.

$$\alpha = \frac{1}{1+\text{decay\_rate}\times\text{epoch\_num}}\alpha_0$$

- α_0: Initial learning rate.
- **decay_rate**: A new hyperparameter to tune.
- **epoch_num**: The current epoch index.

Example Calculation:

- Given: $\alpha_0 = 0.2$, decay_rate $= 1$.
- **Epoch 1:** $\alpha = \frac{1}{1+1\cdot1} \cdot 0.2 = 0.1$
- **Epoch 2:** $\alpha = \frac{1}{1+1\cdot2} \cdot 0.2 \approx 0.067$
- **Epoch 3:** $\alpha = \frac{1}{1+1\cdot3} \cdot 0.2 = 0.05$
- *Result:* The learning rate drops quickly at first, then levels off.

2. Exponential Decay

Lowers the learning rate exponentially based on the epoch.

$$\alpha = (0.95)^{\text{epoch\_num}}\alpha_0$$

(Note: The base, e.g., 0.95, is a hyperparameter less than 1).

3. Square Root Decay

Uses the square root of the epoch or the iteration number.

- **Based on Epoch:** $\alpha = \frac{k}{\sqrt{\text{epoch\_num}}}\alpha_0$
- **Based on Mini-Batch (t):** $\alpha = \frac{k}{\sqrt{t}}\alpha_0$

4. Discrete Staircase Decay

- **Method:** Keep α constant for a specific duration, then drop it largely (e.g., by half) at set intervals.
- *Example:* $\alpha = 0.1$ for epochs 1-10 $\rightarrow$ $\alpha = 0.05$ for epochs 11-20.

5. Manual Decay

- **Method:** Observe the model training over hours or days. When the loss flattens out, manually decrease α.
- **Use Case:** Only practical when training a small number of very large models over long periods.

Strategic Advice

- **Hyperparameter Priority:** Learning rate decay is generally considered **lower priority** compared to tuning the initial learning rate α itself.
- **Recommendation:** Focus on getting a fixed α to work well first. Use decay to squeeze out extra performance or speed up final convergence later.

Optimization Problems: Local Optima vs. Saddle Points

Changing Intuitions

In the early days of Deep Learning, a primary concern was the optimization algorithm getting stuck in **bad local optima**.

- **Old Intuition (Low-Dimensional):** Based on 2-dimensional plots where the cost surface looks like a bumpy landscape with many valleys. In this view, it seems easy to get trapped in a local low point that is not the global minimum.
- **New Intuition (High-Dimensional):** Deep Learning networks operate in extremely high-dimensional spaces (e.g., 20,000+ parameters). The intuition derived from 2D plots does not transfer to these high-dimensional spaces.

Saddle Points

It turns out that **local optima are extremely rare** in high-dimensional neural networks. Most points with zero gradient are actually **Saddle Points**.

- **Definition:** A point where the gradient is zero, but the function curves up in some directions and down in others (resembling a horse saddle).
- **Probability Argument:**
  - For a point to be a **local minimum**, the function must be convex (curve up) in **all** directions.
  - If you have a 20,000-dimensional space, the probability of the function curving up in all 20,000 directions simultaneously is roughly $2^{-20,000}$.
  - It is far more likely that some directions curve up and others curve down, resulting in a saddle point rather than a local optimum.

The Real Problem: Plateaus

If local optima are not the main threat, the real challenge in optimization is **Plateaus**.

- **Definition:** A region where the derivative (gradient) is close to zero for a long period.
- **Impact on Learning:**
  - On a plateau, the gradient is tiny, causing standard Gradient Descent to take extremely small steps.
  - The algorithm can spend a significant amount of time traversing this flat region before finding a slope to descend further.
- **Solution:**
  - This is where advanced algorithms like **Momentum**, **RMSprop**, and **Adam** provide significant value.
  - These algorithms use velocity/momentum to accelerate the traversal across flat plateaus, allowing the network to escape these regions much faster than standard Gradient Descent.

Summary of Optimization Challenges

1. **Bad Local Optima:** Unlikely to be a problem in sufficiently large (high-dimensional) networks.
2. **Plateaus:** A significant problem that slows down convergence. Addressing this requires algorithms with momentum-based components.
